EXERCISE 6
Work with Simple Internal Tables

Unit 2 • ABAP Cloud / ABAP Development Tools

1. Objective

Use iterations and simple internal tables to calculate and display the first 20 Fibonacci numbers.

2. Problem We Solved

The program generates the Fibonacci sequence, stores the values in an internal table, formats every row, and displays the result in the ABAP Console.

Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

3. Task 1 — Calculate the Numbers

Define the number of iterations and the internal table:

```
CONSTANTS max_count TYPE i VALUE 20.
DATA numbers TYPE TABLE OF i.
```

Repeat the calculation and use sy-index to handle each iteration:

```
DO max_count TIMES.

  CASE sy-index.

    WHEN 1.
      APPEND 0 TO numbers.

    WHEN 2.
      APPEND 1 TO numbers.

    WHEN OTHERS.
      APPEND numbers[ sy-index - 2 ]
           + numbers[ sy-index - 1 ]
           TO numbers.

  ENDCASE.

ENDDO.
```

The first iteration stores 0, the second stores 1, and every later iteration adds the two preceding entries.

4. Task 2 — Prepare Formatted Output

```
DATA output TYPE TABLE OF string.

DATA(counter) = 0.
```

```
LOOP AT numbers INTO DATA(number).

  counter = counter + 1.

  APPEND |{ counter WIDTH = 4 ALIGN = LEFT }: { number WIDTH = 10 ALIGN = RIGHT }|
         TO output.

ENDLOOP.
```

5. Task 3 — Write to the Console

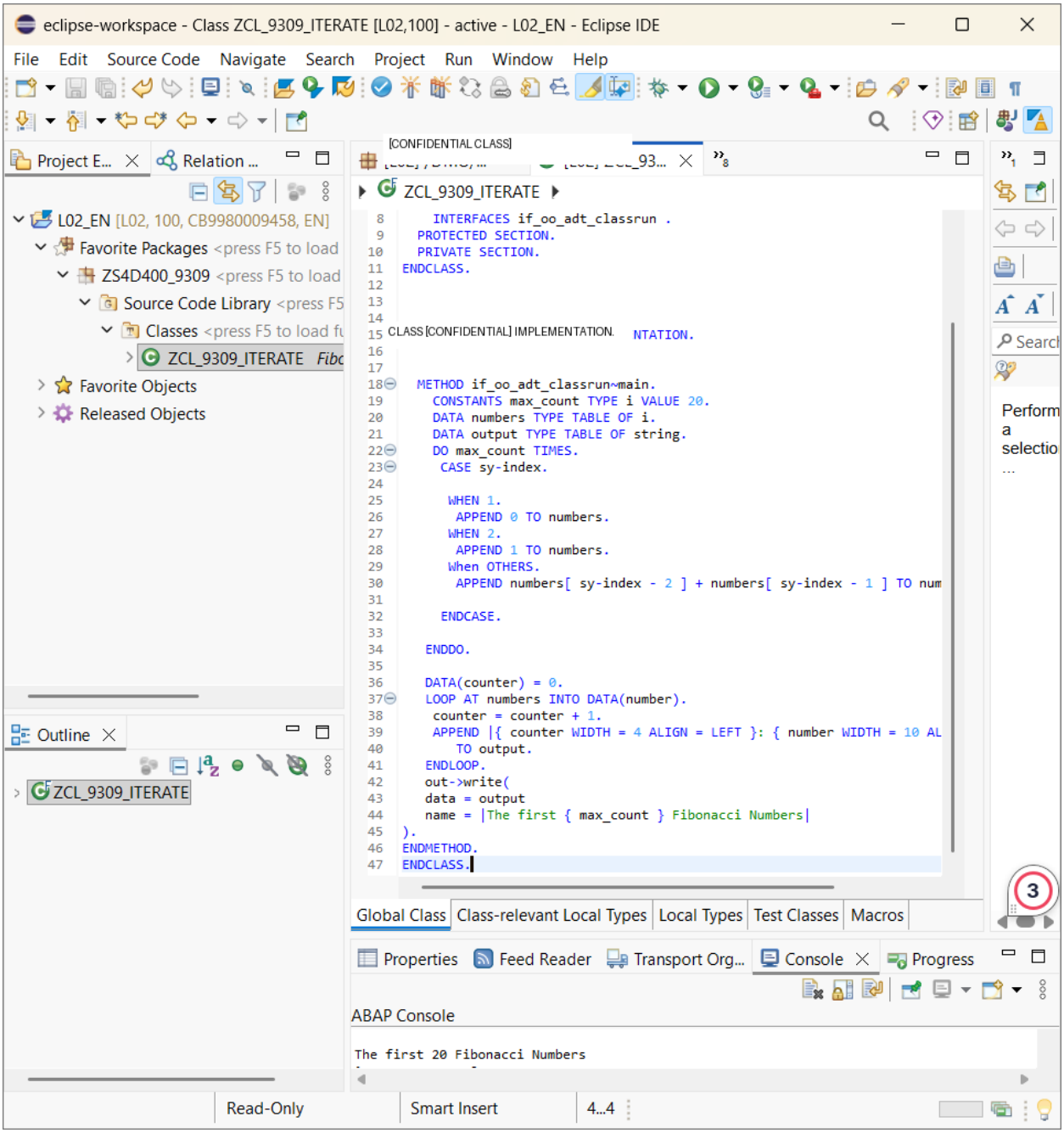
```
out->write(
  data = output
  name = |The first { max_count } Fibonacci Numbers|
).
```

6. Complete Program

```
METHOD if_oo_adt_classrun~main.

  CONSTANTS max_count TYPE i VALUE 20.
  DATA numbers TYPE TABLE OF i.
  DATA output TYPE TABLE OF string.

  DO max_count TIMES.
    CASE sy-index.
      WHEN 1.
        APPEND 0 TO numbers.
      WHEN 2.
        APPEND 1 TO numbers.
      WHEN OTHERS.
        APPEND numbers[ sy-index - 2 ]
             + numbers[ sy-index - 1 ]
             TO numbers.
    ENDCASE.
  ENDDO.

  DATA(counter) = 0.

  LOOP AT numbers INTO DATA(number).
    counter = counter + 1.
    APPEND |{ counter WIDTH = 4 ALIGN = LEFT }: { number WIDTH = 10 ALIGN = RIGHT }|
           TO output.
  ENDLOOP.

  out->write(
    data = output
    name = |The first { max_count } Fibonacci Numbers|
  ).

ENDMETHOD.
```

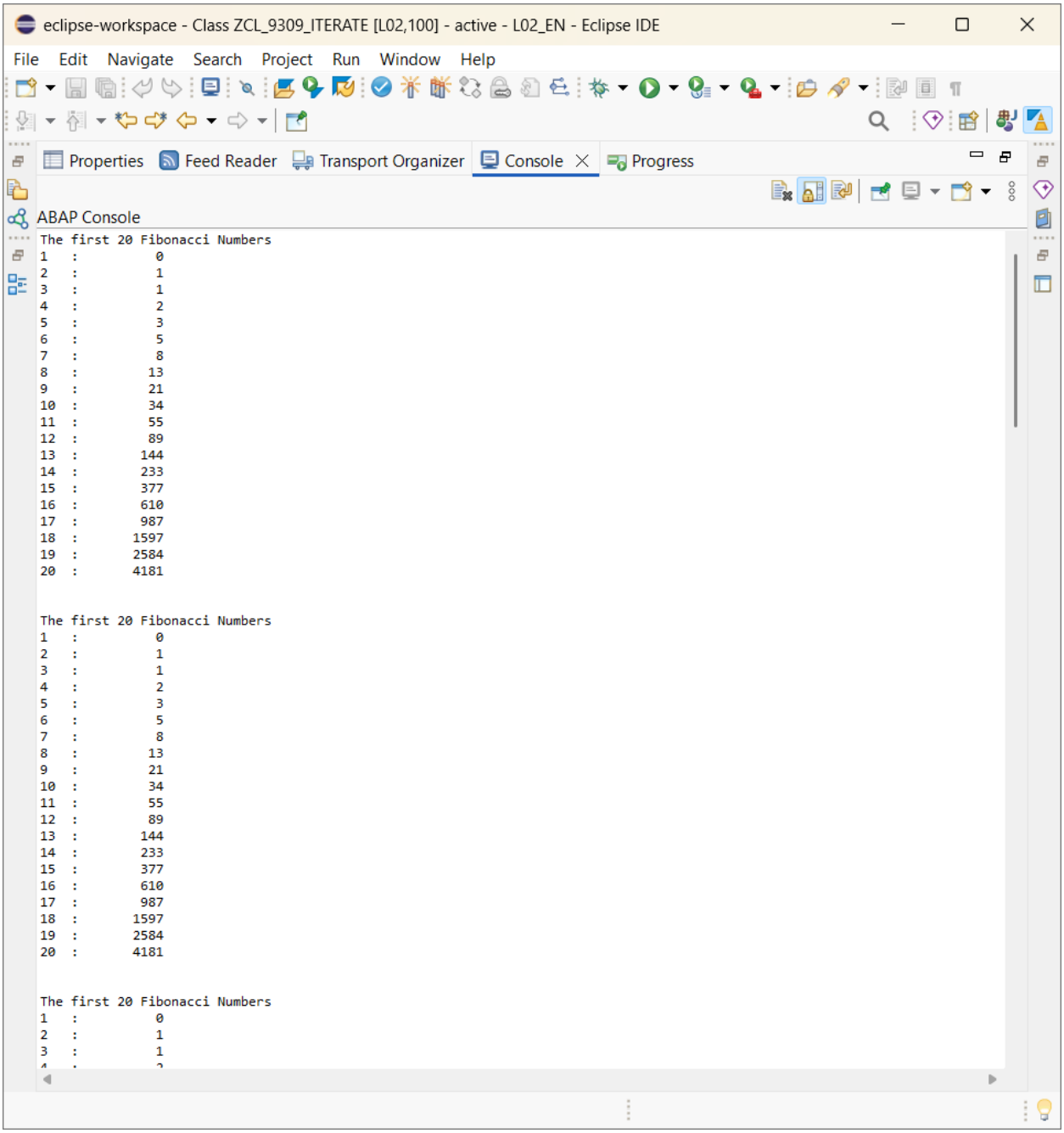
7. Screenshot — Code in Eclipse

This screenshot shows the completed Fibonacci calculation and internal-table processing. The personal/group class identifier has been redacted.



8. Screenshot — Successful Console Output

This screenshot confirms that the program executed successfully and produced the first 20 Fibonacci numbers.



9. Expected Result

```
The first 20 Fibonacci Numbers

1   :          0
2   :          1
3   :          1
4   :          2
5   :          3
6   :          5
7   :          8
8   :         13
9   :         21
10  :         34
```

```
11  :          55
12  :          89
13  :         144
14  :         233
15  :         377
16  :         610
17  :         987
18  :        1597
19  :        2584
20  :        4181
```

10. Why This Matters in a Company

The Fibonacci sequence is only the training problem. The real skills are important in ABAP development because company programs constantly work with internal tables. Developers read database results into internal tables, loop through rows, apply business rules, calculate or transform values, and prepare data for reports, interfaces, APIs, and logs.

11. Activation and Testing

1. Activate the class with Ctrl + F3.
2. Run the console application with F9.
3. Open the Console view.
4. Verify that 20 Fibonacci values are displayed.

12. Confidentiality

Personal/group identifiers, system URLs, usernames, passwords, service keys, transport/access details, and other confidential information are intentionally omitted or redacted from this document.